

Exercícios de Banco de Dados — Resolução

Nome: Luís Eduardo San Martin de Oliveira **RA:** R9186A6 **Prof.:** MSc. Mateus de Paula

1. Banco de dados ou SGBD?

A loja precisa de **um banco de dados**, que é o conjunto organizado de dados sobre vendas e produtos. Esse banco de dados, porém, só existe na prática através de um **SGBD** (Sistema Gerenciador de Banco de Dados) — o software (ex.: PostgreSQL, MySQL) que cria, armazena, protege e permite consultar esses dados. Ou seja: a loja cria um banco de dados **usando** um SGBD; não são alternativas concorrentes, são coisas complementares.

2. Etapas do Projeto de Banco de Dados

1. **Levantamento de requisitos** — entender as necessidades do negócio/usuário.
2. **Projeto conceitual** — criação do modelo Entidade-Relacionamento (independente de SGBD).
3. **Projeto lógico** — conversão do modelo conceitual para o esquema relacional (tabelas, chaves, tipos).
4. **Projeto físico** — implementação no SGBD escolhido (índices, tipos de dado específicos, particionamento).
5. **Implementação e carga de dados.**
6. **Testes e validação.**
7. **Manutenção e evolução** (ao longo do tempo).

3. Propriedades de uma chave primária

- **Unicidade:** não pode haver dois registros com o mesmo valor.
- **Não nulidade (NOT NULL):** nunca pode ser nula.
- **Imutabilidade:** idealmente não deve mudar de valor ao longo do tempo.
- **Minimalidade:** deve usar o menor número possível de atributos para identificar unicamente a linha.

4. Objetivo das chaves estrangeiras

Garantir a **integridade referencial** entre tabelas: uma chave estrangeira impede que um registro referencie um valor que não existe na tabela relacionada, mantendo a consistência dos relacionamentos do modelo.

5. Conversão do modelo conceitual (Aluno - Relacao1 - Curso, N:N)

Como a cardinalidade é **(0,n) para (0,n)** — muitos para muitos —, o relacionamento vira uma **tabela associativa** própria:

```
CREATE TABLE aluno (  
  id_aluno SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL
```

```
);

CREATE TABLE curso (
    id_curso SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE relacao1 (
    id_aluno INT NOT NULL REFERENCES aluno(id_aluno),
    id_curso INT NOT NULL REFERENCES curso(id_curso),
    PRIMARY KEY (id_aluno, id_curso)
);
```

6. Schemas no PostgreSQL

Um *schema* é um **namespace** dentro de um banco de dados: um jeito de agrupar tabelas, views, funções etc. Servem para: - Organizar objetos logicamente (ex.: vendas.pedido, rh.funcionario). - Evitar conflito de nomes entre objetos de times/módulos diferentes. - Controlar permissões de acesso por grupo de objetos.

7. Licença do PostgreSQL

Sim, pode ser distribuído, modificado e usado gratuitamente, inclusive comercialmente. Ele usa a **licença PostgreSQL**, uma licença permissiva semelhante à licença MIT/BSD (sem restrições de copyleft).

9. Empresas que usam PostgreSQL

Instagram, Spotify, Apple, Netflix, Reddit, Skype/Microsoft, IMDb, entre outras.

10. Banco de dados de vendedores/vendas

```
CREATE TABLE vendedor (
    id_vendedor SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    data_nascimento DATE,
    salario NUMERIC(10,2)
);

CREATE TABLE produto (
    id_produto SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE venda (
    id_venda SERIAL PRIMARY KEY,
    id_vendedor INT NOT NULL REFERENCES vendedor(id_vendedor),
    id_produto INT NOT NULL REFERENCES produto(id_produto),
    valor NUMERIC(10,2) NOT NULL
);
```

a) Todos os dados dos vendedores

```
SELECT * FROM vendedor;
```

b) Vendedores organizados pelo nome

```
SELECT * FROM vendedor ORDER BY nome;
```

c) Vendedores com 'na' no nome

```
SELECT nome FROM vendedor WHERE nome ILIKE '%na%';
```

d) Soma das vendas realizadas

```
SELECT SUM(valor) AS total_vendas FROM venda;
```

e) Soma das vendas agrupada por vendedor

```
SELECT id_vendedor, SUM(valor) AS total_vendas  
FROM venda  
GROUP BY id_vendedor;
```

f) Nome e data de nascimento dos vendedores com alguma venda

```
SELECT DISTINCT v.nome, v.data_nascimento  
FROM vendedor v  
JOIN venda ve ON ve.id_vendedor = v.id_vendedor;
```

g) Vendedor que vendeu o produto 'martelo'

```
SELECT v.nome  
FROM vendedor v  
JOIN venda ve ON ve.id_vendedor = v.id_vendedor  
JOIN produto p ON p.id_produto = ve.id_produto  
WHERE p.nome = 'martelo';
```

h) Total de vendedores cadastrados

```
SELECT COUNT(*) AS total_vendedores FROM vendedor;
```

i) Atualizar salário (+100) de quem vendeu mais de 1000 no total

```
UPDATE vendedor  
SET salario = salario + 100  
WHERE id_vendedor IN (  
    SELECT id_vendedor  
    FROM venda  
    GROUP BY id_vendedor  
    HAVING SUM(valor) > 1000  
);
```

11. Junções

a) Junção interna (INNER JOIN)

```
SELECT v.nome, ve.valor  
FROM vendedor v  
INNER JOIN venda ve ON ve.id_vendedor = v.id_vendedor;
```

b) Junção externa à direita (RIGHT JOIN)

```
SELECT v.nome, ve.valor  
FROM vendedor v  
RIGHT JOIN venda ve ON ve.id_vendedor = v.id_vendedor;
```

c) Junção externa à esquerda (LEFT JOIN)

```
SELECT v.nome, ve.valor  
FROM vendedor v  
LEFT JOIN venda ve ON ve.id_vendedor = v.id_vendedor;
```

12. Palavra-chave para linhas únicas

Resposta: c) DISTINCT (no enunciado aparece grafado “DISTINT”, mas é claramente a opção referente a DISTINCT).

13. Comando que NÃO é DML

Resposta: A) ALTER TABLE — é comando de **DDL** (Data Definition Language), não de DML.

14. Médico mais alto (subconsulta com IN)

```
CREATE TABLE medico (  
    id        SERIAL PRIMARY KEY,  
    nome      VARCHAR(50),  
    altura    NUMERIC(3,2)  
);  
  
INSERT INTO medico (id, nome, altura) VALUES  
(1, 'M1', 1.70),  
(2, 'M2', 1.80),  
(3, 'M3', 1.90),  
(4, 'M4', 1.68);  
  
SELECT nome FROM medico  
WHERE altura IN (SELECT MAX(altura) FROM medico);
```

15. Médico mais baixo após novo registro

```
INSERT INTO medico (id, nome, altura) VALUES (5, 'M5', 1.68);  
  
SELECT nome FROM medico  
WHERE altura IN (SELECT MIN(altura) FROM medico);
```

Note que agora existem dois médicos empatados na menor altura (M4 e M5) — a consulta retorna ambos.

16. Médicos com consultas em dezembro/2016

```
CREATE TABLE paciente (  
    id        SERIAL PRIMARY KEY,  
    nome      VARCHAR(50)  
);  
  
INSERT INTO paciente (id, nome) VALUES  
(1, 'P1'), (2, 'P2'), (3, 'P3');  
  
CREATE TABLE consulta (  
    id        SERIAL PRIMARY KEY,  
    idPaciente INT REFERENCES paciente(id),  
    idMedico  INT REFERENCES medico(id),  
    data      DATE  
);  
  
INSERT INTO consulta (id, idPaciente, idMedico, data) VALUES  
(1, 1, 1, '2016-10-10'),  
(2, 2, 1, '2016-12-05'),  
(3, 3, 2, '2017-02-03'),  
(4, 1, 3, '2016-12-15');  
  
SELECT nome FROM medico  
WHERE id IN (  
    SELECT idMedico FROM consulta  
    WHERE data BETWEEN '2016-12-01' AND '2016-12-31'  
);
```

17. Pacientes com mais de 21 anos e consulta marcada

```
ALTER TABLE paciente ADD COLUMN idade INT;

UPDATE paciente SET idade = 10 WHERE id = 1;
UPDATE paciente SET idade = 22 WHERE id = 2;
UPDATE paciente SET idade = 38 WHERE id = 3;

SELECT nome FROM paciente
WHERE idade > 21
AND id IN (SELECT idPaciente FROM consulta);
```

18. Todos os pacientes exceto o mais jovem (predicado ANY)

```
SELECT nome FROM paciente
WHERE idade > ANY (SELECT idade FROM paciente);
```

> ANY retorna quem tem idade maior que **pelo menos um** outro paciente — isso exclui exatamente o mais jovem, que nunca é maior que ninguém.

19. Médicos que ganham mais que todos os pediatras (predicado ALL)

```
ALTER TABLE medico ADD COLUMN salario NUMERIC(10,2);
ALTER TABLE medico ADD COLUMN idEspecialidade INT;

CREATE TABLE especialidade (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(50)
);

INSERT INTO especialidade (id, nome) VALUES
(1, 'Cardiologia'), (2, 'Ortopedia'), (3, 'Pediatria');

ALTER TABLE medico
ADD CONSTRAINT fk_especialidade FOREIGN KEY (idEspecialidade)
REFERENCES especialidade(id);

UPDATE medico SET salario = 1000, idEspecialidade = 3 WHERE id = 1;
UPDATE medico SET salario = 5000, idEspecialidade = 2 WHERE id = 2;
UPDATE medico SET salario = 10000, idEspecialidade = 1 WHERE id = 3;
UPDATE medico SET salario = 12000, idEspecialidade = 3 WHERE id = 4;
UPDATE medico SET salario = 15000, idEspecialidade = 1 WHERE id = 5;

SELECT nome FROM medico
WHERE salario > ALL (
    SELECT salario FROM medico
    WHERE idEspecialidade = (SELECT id FROM especialidade WHERE nome
= 'Pediatria')
);
```

20. Pacientes que já realizaram consulta

```
INSERT INTO paciente (id, nome, idade) VALUES (4, 'P4', 44);
```

a) Com EXISTS

```
SELECT nome FROM paciente p
WHERE EXISTS (
    SELECT 1 FROM consulta c WHERE c.idPaciente = p.id
```

```
);
```

b) Mesma consulta com IN

```
SELECT nome FROM paciente
WHERE id IN (SELECT idPaciente FROM consulta);
```

21. Estados sem fornecedores cadastrados

Resposta: c)

```
SELECT E.nome_estado
FROM Estado AS E
WHERE E.UF NOT IN (
    SELECT F.UF FROM Fornecedor AS F
);
```

A opção **a)** compara nome_estado com UF (tipos/domínios diferentes, errado). As opções **b)** e **d)** têm erro de sintaxe (FROM ... FROM) e não filtram quem não tem fornecedor. A opção **e)** faz o oposto (traz quem TEM fornecedor, com IN em vez de NOT IN).

22. DDL, DML, DQL, DTL e DCL

Sigla	Significado	Função	Exemplo de comando
DDL	Data Definition Language	Define/altera a estrutura dos objetos do banco	CREATE TABLE, ALTER TABLE, DROP TABLE
DML	Data Manipulation Language	Manipula os dados dentro das tabelas	INSERT, UPDATE, DELETE
DQL	Data Query Language	Consulta/recupera dados	SELECT
DTL (DCL de Transação / TCL)	Data Transaction Language (Transaction Control)	Controla transações	COMMIT, ROLLBACK, SAVEPOINT
DCL	Data Control Language	Controla permissões e acesso	GRANT, REVOKE

Exemplos implementáveis no PostgreSQL:

```
-- DDL
CREATE TABLE exemplo_ddl (id SERIAL PRIMARY KEY, nome VARCHAR(50));
ALTER TABLE exemplo_ddl ADD COLUMN ativo BOOLEAN DEFAULT true;
DROP TABLE IF EXISTS tabela_antiga;

-- DML
INSERT INTO exemplo_ddl (nome) VALUES ('Teste');
UPDATE exemplo_ddl SET ativo = false WHERE id = 1;
DELETE FROM exemplo_ddl WHERE id = 1;

-- DQL
SELECT * FROM exemplo_ddl;

-- DTL / TCL
BEGIN;
UPDATE exemplo_ddl SET nome = 'Novo nome' WHERE id = 2;
COMMIT;
-- ou, em caso de erro:
ROLLBACK;
```

```
-- DCL
GRANT SELECT, INSERT ON exemplo_ddl TO usuario_leitura;
REVOKE INSERT ON exemplo_ddl FROM usuario_leitura;
```